



Digital Resume Builder Using Advanced OOP In C++

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

DSA0109 – Object Oriented Programming using C++
to the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by
B. Nikhila (192472102)

Under the Supervision of
Dr. I. Sudha

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105
September 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Digital Resume Builder Using Advanced OOP in C++” has been carried out by B. Nikhila (192472102) of the B.Tech Computer Science and Engineering program under the supervision of Dr. I. Sudha and is submitted in partial fulfilment of the requirements for the current semester of the Bachelor of Technology degree at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. C. Sivasankar

Professor

CSE

Saveetha School of Engineering

SIMATS, Chennai-602105

COURSE FACULTY

Dr. I. Sudha

Professor

CSE

Saveetha School of Engineering

SIMATS, Chennai-602105

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I, B. Nikhila(192472102) of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Digital Resume Builder Using Advanced OOP in C++” is the result of our own bonfide efforts. To the best of our knowledge, the work presented in this report is original and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, software resources, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external software, open-source resources, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that applicable academic, institutional, ethical, and professional requirements have been followed throughout the planning, implementation, testing, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Student

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
P. Tejasri (192472069)	System design and user interface module	C++ implementation – core features	Module testing	Ch. 1 – Introduction, Ch. 3 – System Design	33%
B. Nikhila (192472102)	Resume creation and template management module	C++ implementation – templates and resume generator	Functional testing	Ch. 2 – Literature Study, Ch. 4 – Implementation	34%
V. Supriya (192472194)	File handling, resume storage and data management	C++ implementation – Save, Load, Edit, Delete	Data validation and testing	Ch. 5 – Results & Discussion	33%
Total					100%

ABSTRACT

The Digital Resume Builder Using Advanced OOP in C++ is a menu-driven software application developed to simplify the creation, management, updating, display, and storage of professional resume information. A resume contains several categories of information, including personal details, career objective or summary, education, technical and soft skills, projects, certifications, internships, work experience, achievements, and professional links. Managing these details manually can become repetitive, particularly when information changes frequently or different resume versions are required.

The proposed application addresses this problem by representing major resume sections as classes and objects. A central Resume class coordinates supporting classes such as Personal Info, Education, Skill, Project, Certification, and Experience. Collections such as STL vectors are used for sections containing multiple records. Encapsulation protects internal data, while inheritance and polymorphism can be used for related experience types. File handling provides persistent storage so that resume information can be saved and loaded later.

The system provides operations for creating a resume, adding and updating information, deleting selected records, searching records, displaying a structured resume, and saving or loading data. Input validation and exception handling are included to improve reliability. The implemented solution demonstrates how advanced C++ Object-Oriented Programming concepts can be integrated into a practical, modular, maintainable, and extensible application.

The current implementation is console-based and is primarily intended as an educational and local software application. Future extensions can include a graphical interface, PDF and HTML export, database connectivity, authentication, cloud synchronization, professional templates, and job-specific resume suggestions.

Keywords: C++, Object-Oriented Programming, Resume Builder, File Handling, STL, Software Design

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	
2	CHAPTER 2: Literature Review	
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	
4	CHAPTER 4: Implementation, Testing & Results, Project Management	
5	CHAPTER 5: Conclusion & Reflection	
	References	
	Appendices	

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Stakeholder / User Requirements	12
Table 2	Functional and Non-Functional Requirements	13
Table 3	Comparative Analysis of Existing Approaches	13
Table 4	Design Specifications	14
Table 5	Alternative Solution Evaluation	15
Table 6	Trade-off Analysis	15
Table 7	Sustainability, Ethics, Safety and Security	16
Table 8	Risk Register	17
Table 9	Tools and Software Used	18
Table 10	Test Plan and Verification	20
Table 11	Achievement of Objectives	22
Table 12	Project Roles and Budget	22

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Abbreviation	Description	Unit
OOP	Object-Oriented Programming	—
STL	Standard Template Library	—
I/O	Input / Output	—
C++	C++ Programming Language	—
IDE	Integrated Development Environment	—

CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Digital documents have become an important part of academic and professional life. A resume is one of the most commonly used documents by students, interns, job seekers, and working professionals. It presents qualifications, skills, projects, achievements, experience, and contact information in a structured form. However, when this information changes frequently, manual editing can become time-consuming and may lead to duplication or inconsistency.

C++ is a general-purpose programming language with strong support for object-oriented programming. Classes, objects, access specifiers, inheritance, virtual functions, templates, standard containers, file streams, and exception handling make it suitable for demonstrating how real-world entities can be represented as reusable software components. In this project, the resume is treated as a composite object containing smaller objects such as personal information, education, skills, projects, certifications, and experience.

1.2 Need for the Project

The project is needed because users may repeatedly update academic results, skills, projects, certifications, internships, and work experience. Maintaining these details through unrelated variables or repeated manual editing makes a software solution difficult to extend and maintain. A structured object-oriented approach provides clear ownership of data and operations and supports multiple records for sections such as education, skills, and projects.

The engineering significance of the project is the application of advanced OOP concepts to a realistic information-management problem. The system also provides a practical environment for demonstrating modularity, abstraction, reusability, maintainability, input validation, file persistence, and systematic testing.

1.3 Problem Statement

Students and job seekers require a reliable way to create and maintain structured resume information while frequently adding, modifying, removing, searching, and displaying records. A system that stores all information as unrelated variables becomes difficult to expand, test, and maintain. Therefore, the engineering problem is to design and implement a modular C++ application that models resume information as interacting objects, supports multiple records, validates user input, and provides persistent file-based storage under the constraints of a console-based local application.

1.4 Project Aim

The aim of the project is to design and implement a digital resume builder using advanced C++ Object-Oriented Programming concepts so that resume information can be organized, maintained, displayed, and stored through a simple menu-driven application.

1.5 Project Objectives

The project objectives are to develop a digital resume builder using C++; apply advanced Object-Oriented Programming concepts to a practical application; represent resume sections using independent and reusable classes; provide menu-driven operations for entering and managing information; support multiple education, skill, project, certification, and experience records; provide update, delete, search, and display operations; save and load resume information using file handling; and apply input validation and exception handling to improve reliability.

1.6 Scope

The scope includes personal information, career objective or summary, education, technical and soft skills, projects, certifications, internships, work experience, achievements, professional links, structured resume display, and local file storage. The application is console-based and intended primarily as an educational and local software solution. A graphical user interface, database connectivity, online authentication, cloud synchronization, PDF generation, and web deployment are outside the present implementation scope.

The principal users are students preparing placement and internship resumes, job seekers maintaining professional information, programming learners studying advanced C++ OOP, placement coordinators interested in structured resume collection, and academic evaluators assessing practical OOP implementation.

1.7 Expected Engineering Outcome

The expected outcome is a working console-based digital resume management application in C++. The system should allow users to create or load a resume, manage structured resume sections, validate inputs, display the completed resume, and save information for later use. The engineering outcome is therefore a modular software prototype demonstrating practical application of advanced OOP, STL containers, file handling, and exception handling.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The project review was based on the C++ and Object-Oriented Programming references used in the earlier project report, together with the software concepts needed for class design, STL containers, file input/output, exception handling, modular development, and resume information organization. The reviewed material was used to establish the design concepts and implementation approach rather than to introduce unrelated technologies.

2.2 Review of Existing Work

Existing manual resume preparation commonly uses word processors or document templates. Such approaches are flexible for final formatting, but repeated changes require manual editing of multiple sections. From a software-design perspective, a resume contains several related entities, so object-oriented modeling is appropriate for separating responsibilities and managing collections of records. The earlier project report identified classes such as Resume, PersonalInfo, Education, Skill, Project, Certification, and Experience. It also identified STL vectors for multiple records and file streams for persistence. These concepts form the basis of the proposed system.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual document editing	Flexible formatting and familiar tools	Repeated editing; possible inconsistency	Motivates structured data management
Simple variable-based C++ program	Easy to implement initially	Poor scalability and maintainability	Shows need for class-based design
Object-oriented C++ application	Modular, reusable and maintainable	Console interface in present version	Selected approach
Full production resume platform	GUI, database, cloud and export features	Higher development complexity	Future extension

2.4 Research / Engineering Gap

The identified engineering gap is the need to combine structured resume information management with advanced OOP concepts in a single educational C++ application. A simple data-entry program does not adequately demonstrate modular class design, multiple-record management, persistence, validation, and reusable behavior. The proposed work addresses this gap within the scope of a local console application.

2.5 Proposed Contribution

The proposed contribution is a modular digital resume builder in which major resume sections are represented by classes and coordinated by a central Resume object. The application integrates encapsulation, abstraction, inheritance and polymorphism where appropriate, STL containers for collections, file handling for persistence, and validation for reliable user interaction. The result is both a practical resume-management prototype and a demonstration of advanced C++ OOP.

CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Students	Create and maintain placement or internship resume information.
Job Seekers	Update professional information and maintain structured records.
Placement Coordinators	Support structured collection of student resume information.
Teachers / Evaluators	Assess practical use of advanced C++ OOP concepts.
Developers / Learners	Extend the modular application with additional features.

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Create resume profile	Functional	A new resume can be created through the menu.
Add / update / delete records	Functional	Records can be managed without restarting the application.
Search records	Functional	Relevant resume records can be located.
Display structured resume	Functional	Complete entered information is displayed in an organized sequence.

Save and load data	Functional	Resume information can be stored and retrieved from a file.
Usability	Non-Functional	Menu options and messages remain clear.
Reliability	Non-Functional	Invalid input is handled without unexpected termination.
Maintainability	Non-Functional	Responsibilities are separated into logical classes.
Portability	Non-Functional	Program compiles using a standard C++ compiler.

3.2 Constraints & Applicable Standards

The applicable constraints are primarily software and project-scope constraints. The current system is console-based, uses local file storage, and is intended for a standard C++ environment on Windows. The project report identifies C++23 and recognized software-quality practices as relevant references. The source report does not provide evidence that a separate product-specific regulatory standard was applied, so no unsupported standard is claimed here.

Standard / Code	Requirement	How Applied in This Project
ISO/IEC C++ Standard – C++23 reference	Use standard C++ language and library concepts.	C++ classes, STL containers, streams and language features are used.
Structured software development practices	Separate requirements, design, implementation and testing.	The report follows a defined engineering lifecycle.
Engineering ethics / academic integrity	Use original work and acknowledge external resources.	Declaration and references are included.

3.3 Design Specifications

Parameter	Target Requirement /	Unit	Priority
Resume information model	Separate classes for major resume entities	classes	High
Multiple records	Support multiple education, skill, project and related entries	records	High
Persistence	Save and load resume information	file	High
Validation	Handle invalid menu and data inputs	operation	High
Interface	Clear menu-driven console interaction	interface	Medium
Maintainability	Modular responsibilities	design	High

3.4 Alternative Solutions & Evaluation

Alternative 1: A simple variable-based C++ program stores all resume fields directly in the main program. It is easy to start but becomes difficult to extend when multiple records and new sections are added.

Alternative 2: The selected object-oriented C++ design models each resume section as a class and uses a central Resume object. It requires more initial design effort but provides stronger modularity, reuse and maintainability.

Alternative 3: A database-backed web or GUI application could provide richer user interaction and multi-user storage, but it is beyond the present course-focused console application scope.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	20%	Good	Good	Good
Cost / Complexity	20%	Low	Medium	High
Safety / Reliability	20%	Medium	High	High
Sustainability / Maintainability	20%	Low	High	High
Scalability	20%	Low	High	Very High

3.5 Selected Design & Detailed Design

The object-oriented C++ design is selected because it directly satisfies the project's principal requirement: to demonstrate advanced OOP while solving the resume-management problem. The selected design separates information into meaningful classes and uses the Resume class as the central coordinator. This structure makes it easier to add new sections without redesigning the entire program.

The user interacts with the menu and the Resume controller. The controller coordinates PersonalInfo, Education, Skill, Project, Certification and Experience objects. Multiple records can be stored using STL containers. File storage is connected to the resume data so that information can be saved and loaded. The design therefore follows a system-level separation of responsibilities.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Object-oriented class model	Single large function / variable set	Modularity and maintainability	More initial design effort	Selected because it directly demonstrates advanced OOP.
File-based persistence	Database	Simple local implementation	Less suitable for multi-user	Selected because database connectivity is

			scaling	outside current scope.
Console interface	GUI / Web interface	Low complexity and suitable for course implementation	Less visual and user-friendly	Selected for current scope; GUI is future work.
STL containers	Fixed-size arrays	Flexible management of multiple records	Requires understanding of STL	Selected for reusable collection management.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Prefer modular software that can be extended without replacing the whole system.	Class-based architecture and reusable components.	Modular OOP design was selected.
Ethics	Maintain academic integrity and acknowledge external resources.	Declaration and reference list.	External references and tools are acknowledged.
Health & Safety	Software-only project has no physical hazards.	Console application and local computer use.	No hardware safety risk identified.
Security	Resume data can contain personal contact information.	File-based local storage and project scope.	Future production versions should add protected storage and authentication.

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Invalid user input	Medium	Medium	Medium	Input validation and exception handling	Low
File unavailable / corrupted	Medium	Medium	Medium	Check file operations and use consistent storage format	Low–Medium
Loss of unsaved changes	Medium	Medium	Medium	Provide explicit save operation and clear user messages	Low–Medium
Exposure of resume information	Low	High	Medium	Restrict production deployment and use secure storage in future	Low–Medium
Poor maintainability if features are added without structure	Medium	Medium	Medium	Maintain separation of class responsibilities	Low

CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The development approach begins with requirement analysis and identification of the information that a professional resume should contain. The identified entities are then converted into classes and their relationships are defined. Implementation converts the design into C++ code using constructors, member functions, access control, inheritance and virtual functions where appropriate. Validation, file persistence, testing and evaluation are then applied to confirm that the implemented workflow satisfies the project objectives.

Resource	Purpose
C++ compiler / standard library	Program development and execution
Antigravity IDE	Code development and project editing
Windows operating system	Development and execution environment
Text / binary file streams	Local persistence of resume data
STL containers	Management of multiple records

4.2 Software Implementation & Modern Tools Used

The implementation is a menu-driven C++ application. The source report identifies C++ as the programming language, Advanced Object-Oriented Programming as the main programming approach, Antigravity IDE as the development environment, file handling as the storage mechanism, and Windows as the operating system. The application separates resume data into logical classes and uses file streams to preserve information.

Tool / Software / Equipment	Purpose	Stage Used
C++	Application implementation	Design and implementation
Advanced OOP	Class modeling and modular design	Design and implementation
Antigravity IDE	Writing and testing the C++ program	Implementation

STL vector / string	Managing collections and text	Implementation
fstream	Saving and loading resume data	Implementation
Windows	Development and execution	All stages

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Create resume	Run program and enter valid resume data	Resume profile is created successfully.
Add records	Enter education, skills, projects and related data	Entered records are stored in the appropriate sections.
Update / delete	Modify and remove selected information	Selected information changes or is removed correctly.
Search	Search for an existing record	Matching record is located.
Display	Select display option after data entry	Resume is shown in structured order.
Save / load	Save data, restart or reload, then load file	Saved information is retrieved correctly.
Invalid input	Enter unsupported menu or data values	Program handles the input without unexpected termination.

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Resume creation	Supported	Supported according to implementation	Pass
Structured class model	Required	Implemented	Pass
File persistence	Required	Implemented	Pass

Input validation	Required	Implemented / considered	Pass
Menu-driven operation	Required	Implemented	Pass

4.4 Results

The project report states that the Digital Resume Builder was evaluated through its major functions, including creating a resume, entering personal information, adding education details, skills, projects, certifications and experience, and performing update, delete, search and display operations. The evaluation showed that the application organized entered information through classes and objects and displayed the complete resume in a structured format.

File handling was tested by saving resume information and loading it again when required. Input validation and exception handling were also considered to reduce unexpected program termination. Based on the documented evaluation, the system meets its main objectives and demonstrates the intended C++ OOP concepts.

4.5 Data Analysis & Engineering Judgment

The principal engineering judgment is based on functional verification rather than numerical physical measurements because this is a software-only project. The documented results indicate that the chosen class-based architecture successfully organizes different resume entities, while STL containers support multiple records and file streams provide persistence. The main engineering implication is that modular object-oriented design is more suitable than a collection of unrelated variables when the application is expected to grow.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Develop a digital resume builder using C++	Implemented menu-driven C++ application	Fully Achieved
Apply advanced OOP concepts	Classes, encapsulation, inheritance, polymorphism and abstraction	Fully Achieved

Support reusable resume sections	Separate logical classes and collections	Fully Achieved
Support add / update / delete / search / display	Documented application operations	Fully Achieved
Save and load information	File stream implementation	Fully Achieved
Apply validation and exception handling	Validation and exception-handling approach documented	Achieved within current scope

4.7 Strengths & Limitations

Strengths include modular class design, practical demonstration of advanced OOP, support for multiple records, file-based persistence, structured resume display, and a clear menu-driven workflow. The design also provides a foundation for future GUI, database, export and cloud features.

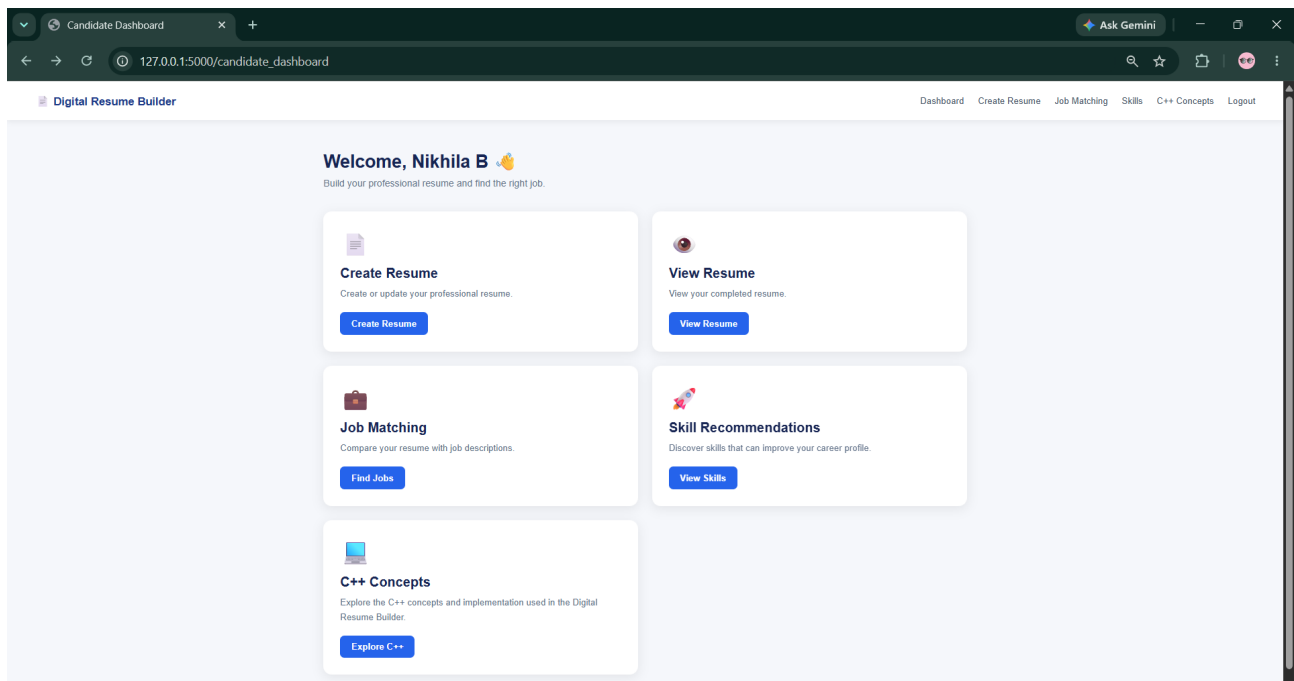
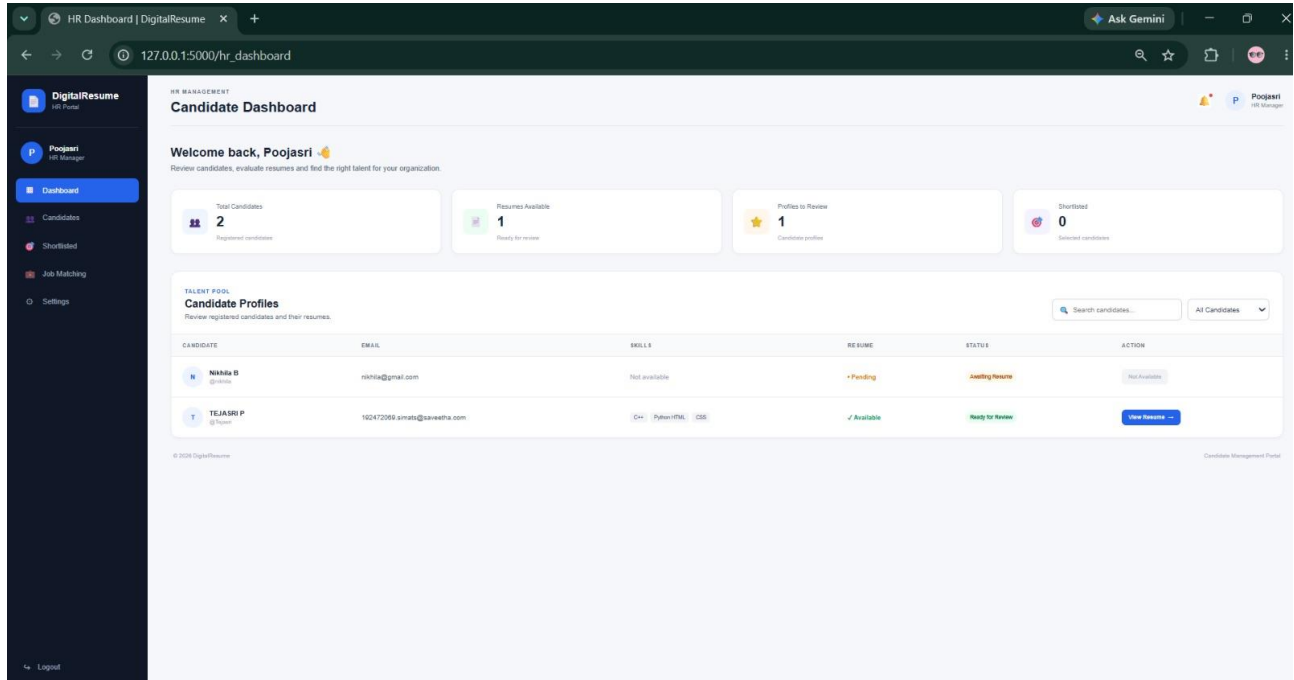
Limitations include the console-only interface, local file storage instead of a database, absence of authentication, absence of cloud synchronization, and limited final-document formatting compared with professional resume platforms. The current project is therefore a functional educational prototype rather than a complete production resume service.

4.8 Project Planning, Roles & Budget

The source report does not provide a completed Gantt chart, actual individual role allocation, or verified project expenditure. To avoid inventing project records, the following planning and budget tables are provided as clean fields to be completed from the team's actual records.

Phase	Milestone
Requirement Analysis	Identify resume information and user operations
Object-Oriented Design	Define classes, relationships and data structures
Implementation	Develop menu, classes, operations and file handling

Testing	Verify functional workflows and invalid-input handling
Evaluation	Compare results with objectives and document limitations
Documentation	Prepare final capstone report and appendices



CHAPTER 5: CONCLUSION & REFLECTION

5.1 Conclusion

The Digital Resume Builder Using Advanced OOP in C++ provides a structured solution for creating and managing professional resume information. The project models personal, academic, technical, project, certification and professional information through object-oriented classes. The implementation demonstrates how encapsulation, abstraction, inheritance, polymorphism, STL containers, file handling, constructors, destructors and exception handling can be combined in a practical C++ application.

The documented testing confirms the main workflow of creating, managing, displaying, saving and loading resume information. The project therefore achieves its primary educational and engineering purpose while providing a foundation for further development.

5.2 Future Work

Future development can extend the application with a graphical user interface, professional PDF and HTML export, multiple resume templates, database integration, multiple user profiles and login, cloud synchronization, secure online sharing, job-role-based keyword suggestions, spell checking, grammar assistance, multilingual support, customizable sections, and job-description analysis. AI-assisted resume suggestions and placement-management integration can also be considered as future extensions.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

The project strengthened understanding of advanced C++ and Object-Oriented Programming by requiring theory to be applied to a complete software problem. The team gained practical understanding of classes and objects, access control, constructors and destructors, inheritance, polymorphism, abstraction, STL containers, file input/output, validation, exception handling and modular code organization.

Engineering & Professional Skills Developed

The project developed skills in requirement analysis, object modeling, modular software design, debugging, test-case preparation, file management and technical documentation. It also reinforced the importance of separating responsibilities so that future changes can be made without unnecessarily affecting unrelated modules.

Individual Reflection

P. Tejasri (192472069): The individual reflection should describe the most difficult engineering problem encountered, the key engineering decision personally made and the evidence supporting it, the new knowledge acquired independently, and what would be redesigned if the project were repeated. The specific personal contribution is to be completed from the team's actual work record.

B. Nikhila (192472102): The individual reflection should describe the most difficult engineering problem encountered, the key engineering decision personally made and the evidence supporting it, the new knowledge acquired independently, and what would be redesigned if the project were repeated. The specific personal contribution is to be completed from the team's actual work record.

V. Supriya (192472194): The individual reflection should describe the most difficult engineering problem encountered, the key engineering decision personally made and the evidence supporting it, the new knowledge acquired independently, and what would be redesigned if the project were repeated. The specific personal contribution is to be completed from the team's actual work record.

REFERENCES

1. Horton, I., & Van Weert, P. (2023). Beginning C++23: From Beginner to Pro. Apress.
2. Somashekara, M. T., Guru, D. S., Nagendraswamy, H. S., & Manjunatha, K. S. (2023). Object-Oriented Programming with C++. PHI Learning.
3. Narula, H. G., Shah, K., Nasir, A. M., Pednekar, D. R., & Joshi, P. A. (2023). Object Oriented Programming with C++. TechKnowledge Publications.
4. Wadhwa, S. (2023). Object Oriented Programming Using C++. EPEX Publications.
5. Bhawe, V. S., & Patekar, A. (2024). Object Oriented Programming with C++, 2nd Edition. Pearson India.
6. Thareja, R. (2025). Object Oriented Programming with C++, 2nd Edition. Oxford University Press India.
7. Balagurusamy, E. (2026). Object Oriented Programming with C++, 9th Edition. McGraw Hill Education.
8. cppreference.com. (2023). C++23 Language and Standard Library Reference.
9. ISO/IEC. (2023). Programming Languages — C++ Standard: C++23. International Organization for Standardization.
10. Microsoft. (2024–2026). C++ Documentation and Standard Library Reference. Microsoft Learn.

APPENDICES

Appendix A – Detailed Calculations

Not applicable to the present software-only project. No physical engineering derivations are required. Software design and verification evidence are included in Chapters 3 and 4.

Appendix B – Engineering Drawings / Schematics

The system workflow and architecture diagrams are included in Chapter 1 and Chapter 3.

Appendix C – Source Code / Code Repository Information

The source report contains the core C++ implementation. Only essential code excerpts should normally be included in the final report; the complete source code may be maintained separately in the approved repository.

```
#include <iostream>
#include <vector>
#include <fstream>
using namespace std;

class Person {
protected:
    string name, email;
public:
    void input() {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Email: ";
        cin >> email;
    }

    virtual void display() {
        cout << "Name : " << name << endl;
        cout << "Email : " << email << endl;
    }
};
```

```

class Resume : public Person {
private:
    string skill;
    string project;
public:
    void input() {
        Person::input();
        cout << "Enter Skill: ";
        cin >> skill;
        cout << "Enter Project: ";
        cin >> project;
    }
    void display() override {
        cout << "\n===== DIGITAL RESUME =====\n";
        Person::display();
        cout << "Skill   : " << skill << endl;
        cout << "Project : " << project << endl;
    }
    void save() {
        ofstream file("resume.txt");
        file << name << endl;
        file << email << endl;
        file << skill << endl;
        file << project << endl;
        file.close();
        cout << "\nResume saved successfully!";
    }
};

int main() {
    Resume r;
    r.input();
    r.display();
}

```

```
r.save();  
return 0;  
}
```

Appendix D – Datasheets

Not applicable to this software-only project.

Appendix E – Experimental / Raw Data

The project is evaluated through software test cases rather than physical measurements. The test plan and verification tables are included in Chapter 4.

Appendix F – Ethics / Institutional Approval

The project uses no human-subject experiment or physical prototype. The declaration and academic-integrity statement are included in the front matter.

Appendix G – Project Meeting / Progress Records

To be attached if maintained by the team.

Appendix H – User / Industry Feedback

To be attached if collected.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None reported in the source project report.

Appendix J – Individual Contribution Evidence

Team contribution evidence, screenshots, task records or other supporting material may be attached here. The exact evidence should reflect the team's actual work.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

The following mapping follows the new capstone template. The location column identifies where the relevant evidence appears in this report.

Outcome / Area	Evidence	SO / Area	Location in This Report
Engineering problem and solution	Problem statement, aim, objectives and selected design	SO1	Ch. 1 and Ch. 3.5
Standards	C++23 reference and structured software practices	SO2	Ch. 3.2
Sustainability and societal / environmental impact	Modularity and maintainability considerations	SO2 / SO4	Ch. 3.7
Ethics and professional responsibility	Academic integrity, acknowledgement and responsible handling of personal data	SO4	Declaration and Ch. 3.7
Risk assessment and mitigation	Risk register	SO2 / SO4	Ch. 3.7
Security considerations	Resume-data privacy and future protected storage	Relevant SO	Ch. 3.7
Integrated / system-level engineering	Architecture and subsystem relationships	SO1 / SO2	Ch. 3.5
Experimentation, analysis and engineering judgment	Test plan, verification and interpretation	SO6	Ch. 4.3–4.6
Project planning and management	Milestones, roles and budget fields	SO5	Ch. 4.8
Independent acquisition of new knowledge	Learning and technical skills	SO7	Ch. 5.3
Individual contribution	Contribution statement and supporting evidence	SO1–SO7	Contribution Statement + Ch. 4.8

